

Softwares de Sistemas

Java Virtual Machine

SOFTWARE BÁSICO, TURMA A
PROF. MARCELO LADEIRA CIG/UNB

Baseado no livro "The Java Virtual Machine Specification", Second Edition, Tim Lindholm & Frank Yellin. Parte 2 de 3

Interpretador

Instrução da JVM

- Ignorando a execução de exceções, o ciclo interno de um interpretador da JVM pode ser descrito como;

```
do {  
    busque um opcode;  
    if ( $\exists$  operandos) busque operandos;  
    execute a ação associada ao opcode;  
} while (! fim);
```

Conjunto de Instruções

Instrução da JVM

- um byte de *opcode* especificando a operação
 - zero ou mais *operandos* que representam argumentos ou dados que serão usados na operação.
 - o número e tamanho dos operandos são determinados pelo opcode.
 - se o **operando é maior que um byte** em tamanho, é armazenado em **big-endian ordem** (byte mais significativo primeiro).
 - para construir operando de 16 bits use $(\text{byte1} \ll 8) \mid \text{byte2}$
- alinhadas por byte
 - Exceto **tableswitch** (que acessa tabela de saltos por índice e desvia) e **lookupswitch** (que acessa tabela de saltos por batimento de chave e desvia)
 - utilizam de **0 a 3** bytes (NULOS) como preenchimento (padding)
 - alinha o primeiro operando em **fronteira de 4 bytes**

Qual a razão de apenas um byte para código de operação? Como a JVM sabe quais os tipos dos operandos?

tableswitch

OxAA

0 a 3 bytes

32 bits signed

32 bits signed

32 bits signed

x offsets

/* opcode */

/* alinha bytes a seguir em múltiplos de 4 a partir daqui */

/* default – usado para calcular *target* default */

/* low – índice inferior da tabela de offsets */

/* high – índice superior da tabela de offsets */

/* x = high-low+1 offsets de 32 bits com sinal */

■ Comentários

- use $(\text{byte1} \ll 24) \mid (\text{byte2} \ll 16) \mid (\text{byte3} \ll 8) \mid \text{byte4}$ para construir um offset de 32 bits com sinal. Os bytes são considerados sem sinais.
- operação na pilha de operandos
..., *index* $\Rightarrow$...
- se *index* < *low* ou *index* > *high* então *target* = *default* + offset do opcode da instrução *tableswitch*
- senão *target* = *offset[index-low]* + offset do opcode da instrução *tableswitch*
- *target* é um offset em relação ao MÉTODO que contém a instrução *tableswitch*

index é do tipo inteiro. Os cases podem ser representados como uma tabela de forma eficiente, não esparsa.

tableswitch

..., index $\Rightarrow$...

tableswitch opcode

<0-3 byte pad> /* alinha operandos a seguir em fronteira de 4 bytes a partir do início do método */

defaultbyte1

defaultbyte2

defaultbyte3

defaultbyte4

lowbyte1

lowbyte2

lowbyte3

lowbyte4

highbyte1

highbyte2

highbyte3

highbyte4

jump offsets...

Próxima instrução a ser executada tem endereço target + endereço da instrução atual.

tableswitch

```
int chooseNear(int i) {
  switch (i) {
    case 0: return 0;
    case 1: return 1;
    case 2: return 2;
    default: return -1;
  }
}
```

```
Method int chooseNear(int)
0 iload_1          // Push local variable 1 (argument i)
1 tableswitch 0 to 2: // Valid indices are 0 through 2
0: 28              // If i is 0, continue at 28
1: 30              // If i is 1, continue at 30
2: 32              // If i is 2, continue at 32
default: 34        // Otherwise, continue at 34
28 iconst_0        // i was 0; push int constant 0...
29 ireturn         // ...and return it
30 iconst_1        // i was 1; push int constant 1...
31 ireturn         // ...and return it
32 iconst_2        // i was 2; push int constant 2...
33 ireturn         // ...and return it
34 iconst_m1       // otherwise push int constant -1...
35 ireturn         // ...and return it
```

170:	0A	00	01	00	0B	00	00	00	4C	00	01	00	01	00	00	00	L.....
180:	24	1A	AA	00	00	00	00	00	21	00	00	00	00	00	00	00	\$.a.....!
190:	02	00	00	00	1B	00	00	00	1D	00	00	00	1F	03	AC	04	~.....
1A0:	AC	05	AC	02	AC	00	00	00	01	00	0C	00	00	00	16	00	~.~.~.....
1B0:	05	00	00	00	06	00	1C	00	07	00	1E	00	08	00	20	00	
1C0:	09	00	22	00	0A	00	01	00	0D	00	0E	00	01	00	0B	00	.."......
1D0:	00	00	2D	00	02	00	01	00	00	00	0D	2A	B7	00	01	B2	..-.....*...²
1E0:	00	02	12	03	B6	00	04	B1	00	00	00	01	00	0C	00	00	9...±.....
1F0:	00	0E	00	03	00	00	00	0D	00	04	00	0E	00	0C	00	0F	
200:	00	09	00	0F	00	10	00	01	00	0B	00	00	00	5F	00	02	
210:	00	01	00	00	00	33	B2	00	02	02	B8	00	05	B6	00	06	3²....9....

33 (21h) – default
 00 – low
 02 – high
 0 ~ 27 (1Bh)
 1 ~ 29 (1Dh)
 2 ~ 31 (1Fh)

lookupswitch

0xAB	/* opcode */
0 a 3 bytes	/* alinha bytes a seguir em múltiplos de 4 a partir daqui */
32 bits signed	/* default – usado para calcular <i>target</i> default */
32 bits signed	/* npairs – número de pares da tabela de offset */
npairs pares	/* cada par tem a forma <match><offset> */

■ Comentários

- use $(\text{byte1} \ll 24) \mid (\text{byte2} \ll 16) \mid (\text{byte3} \ll 8) \mid \text{byte4}$ para construir um valor de 32 bits. Os bytes são considerados sem sinais.
- *npairs* é inteiro maior ou igual a zero. *match* e *key* são do tipo inteiros.
- a tabela de pares é ordenada por *match* crescente.
- operação na pilha de operandos
..., *key* $\Rightarrow$...
- se *key* não casa com nenhum *match* então *target* = *default* + offset do opcode da instrução *lookupswitch*
- senão *target* = offset associado ao *match* casado + offset do opcode da instrução *lookupswitch*
- *target* é um offset em relação ao MÉTODO que contém a instrução *lookupswitch*

lookupswitch

..., key $\Rightarrow$...

lookupswitch opcode

<0-3 byte pad> /* alinha operandos a seguir em fronteira de 4 bytes */

defaultbyte1

defaultbyte2

defaultbyte3

defaultbyte4

npairs1

npairs2

npairs3

npairs4

pares match-offset

- são pares de 4 bytes cada, ordenados por match

Próxima instrução a ser executada tem endereço target + endereço da instrução atual.

lookupswitch

```
int chooseFar(int i) {
```

```
    switch (i) {
```

```
        case -100: return -1;
```

```
        case 0: return 0;
```

```
        case 100: return 1;
```

```
        default: return -1;
```

```
    }
```

```
}
```

```
Method int chooseFar(int i)
```

```
0 iload_1
```

```
// Push local variable 1 (argument i)
```

```
1 lookupswitch 3:
```

```
// Valid indices are -100, 0, and 100
```

```
-100: 36
```

```
// If i is -100, continue at 36
```

```
0: 38
```

```
// If i is 0, continue at 38
```

```
100: 40
```

```
// If i is 100, continue at 40
```

```
default: 42
```

```
// Otherwise, continue at 42
```

```
36 iconst_m1
```

```
// i was -100; push int constant -1...
```

```
37 ireturn
```

```
// ...and return it
```

```
38 iconst_0
```

```
// i was 0; push int constant 0...
```

```
39 ireturn
```

```
// ...and return it
```

```
40 iconst_1
```

```
// i was 100; push int constant 1...
```

```
41 ireturn
```

```
// ...and return it
```

```
42 iconst_m1
```

```
// otherwise push int constant -1...
```

```
43 ireturn
```

```
// ...and return it
```

```
180: 00 2E 1A AB 00 00 00 00 00 29 00 00 00 03 FF FF
190: FF 9C 00 00 00 23 00 00 00 00 00 00 00 25 00 00
1A0: 00 64 00 00 00 27 02 AC 03 AC 04 AC 11 00 C8 AC
1B0: 00 00 00 01 00 0C 00 00 00 16 00 05 00 00 00 06
1C0: 00 24 00 07 00 26 00 08 00 28 00 09 00 2A 00 0A
1D0: 00 01 00 0D 00 0E 00 01 00 0B 00 00 00 2D 00 02
1E0: 00 01 00 00 00 0D 2A B7 00 01 B2 00 02 12 03 B6
```

41 (29h) – default

03 – npairs

-100 ~ 35 (FFFFFF9Ch)

0 ~ 37 (0h)

100 ~ 39 (64h)

Conjunto de Instruções

Tipos e a JVM

- Em geral o tipo dos operandos é explicitado no mnemônico da instrução por uma letra de prefixo
 - i* *l* *s* *b* *c* *f* *d*
int *long* *short* *byte* *char* *float* *double*
 - a*
representa o tipo de referência
- Instruções com tipo não ambíguo não possuem prefixo de tipo
 - por exemplo, *arraylength* opera em objeto do tipo *array*
..., *arrayref* $\Rightarrow$..., *length*
- Instruções que não operam em operandos com tipos
 - por exemplo, *goto*
goto branchbyte1 branchbyte2
... $\Rightarrow$...

O conjunto de instruções da JVM não é ortogonal. Se fosse, apenas um byte de opcode seria suficiente?

Conjunto de Instruções

Template de instrução com tipo

- Uma instrução específica com informação de tipo é construída substituindo o T no template pela letra na coluna de tipo.
 - se a célula determinada por um coluna de tipo e uma linha de *template* for vazia, não existe instrução que suporte aquele tipo de operação.
 - por exemplo, load de int é iload. Não existe um load de byte.
 - para a maioria das instruções não existe uma forma para os tipos integrais byte, char e short.
 - não existe instrução com operando boolean
 - byte, e short são tratados como int com extensão de sinal
 - char e boolean são tratados como int com extensão de zero

Sumário do Conjunto de Instruções

<i>opcode</i>	<i>byte</i>	<i>short</i>	<i>int</i>	<i>long</i>	<i>float</i>	<i>double</i>	<i>char</i>	<i>reference</i>
<i>Tipush</i>	<i>bipush</i>	<i>sipush</i>						
<i>Tconst</i>			<i>iconst</i>	<i>lconst</i>	<i>fconst</i>	<i>dconst</i>		<i>aconst</i>
<i>Tload</i>			<i>iload</i>	<i>lload</i>	<i>float</i>	<i>dload</i>		<i>aload</i>
<i>Tstore</i>			<i>istore</i>	<i>lstore</i>	<i>fstore</i>	<i>dstore</i>		<i>astore</i>
<i>Tinc</i>			<i>iinc</i>					
<i>Taload</i>	<i>baload</i>	<i>saload</i>	<i>iaload</i>	<i>laload</i>	<i>faload</i>	<i>daload</i>	<i>caload</i>	<i>aaload</i>
<i>Tastore</i>	<i>bastore</i>	<i>sastore</i>	<i>iastore</i>	<i>lastore</i>	<i>fastore</i>	<i>dastore</i>	<i>castore</i>	<i>aastore</i>
<i>Tadd</i>			<i>iadd</i>	<i>ladd</i>	<i>fadd</i>	<i>dadd</i>		
<i>Tsub</i>			<i>isub</i>	<i>lsub</i>	<i>fsub</i>	<i>dsub</i>		
<i>Tmul</i>			<i>imul</i>	<i>lmul</i>	<i>fmul</i>	<i>dmul</i>		
<i>Tdiv</i>			<i>idiv</i>	<i>ldiv</i>	<i>fdiv</i>	<i>ddiv</i>		
<i>Trem</i>			<i>irem</i>	<i>lrem</i>	<i>frem</i>	<i>drem</i>		
<i>Tneg</i>			<i>ineg</i>	<i>lneg</i>	<i>fneg</i>	<i>dneg</i>		
<i>Tshl</i>			<i>ishl</i>	<i>lshl</i>				

Sumário do Conjunto de Instruções

<i>opcode</i>	<i>byte</i>	<i>short</i>	<i>int</i>	<i>long</i>	<i>float</i>	<i>double</i>	<i>char</i>	<i>reference</i>
<i>Tshr</i>			<i>ishr</i>	<i>lshr</i>				
<i>Tushr</i>			<i>iushr</i>	<i>lushr</i>				
<i>Tand</i>			<i>iand</i>	<i>land</i>				
<i>Tor</i>			<i>ior</i>	<i>lor</i>				
<i>Txor</i>			<i>ixor</i>	<i>lxor</i>				
<i>i2T</i>	<i>i2b</i>	<i>i2s</i>		<i>i2l</i>	<i>i2f</i>	<i>i2d</i>		
<i>l2T</i>			<i>l2i</i>		<i>l2f</i>	<i>l2d</i>		
<i>f2T</i>			<i>f2i</i>	<i>f2l</i>		<i>f2d</i>		
<i>d2T</i>			<i>d2i</i>	<i>d2l</i>	<i>d2f</i>			
<i>Tcmp</i>				<i>lcmp</i>				
<i>Tcmpl</i>					<i>fcmpl</i>	<i>dcmpl</i>		
<i>Tcmpg</i>					<i>fcmpg</i>	<i>dcmpg</i>		
<i>if_TcmpOP</i>			<i>if_icmpOP</i>					<i>if_acmpOP</i>
<i>Treturn</i>			<i>ireturn</i>	<i>lreturn</i>	<i>freturn</i>	<i>dreturn</i>		<i>areturn</i>

Mapeamento dos Tipos de Java na JVM

Java	JVM	Categoria
boolean	int	1
byte	int	1
char	int	1
short	int	1
int	int	1
float	float	1
reference	reference	1
	returnAddress	1
long	long	2
double	double	2

Instruções e Tipos de Operandos

Normalmente o opcode determina o tipo dos operandos

- algumas instruções operam na pilha limitadas por categoria e não por tipo

pop

..., *value* $\Rightarrow$... /* *value* deve ser da categoria 1 */

pop2

..., *value2*, *value1* $\Rightarrow$... /* *value2* e *value1* categoria 1 */

..., *value* $\Rightarrow$... /* *value* deve ser da categoria 2 */

swap

..., *value2*, *value1* $\Rightarrow$..., *value1*, *value2* /* somente categoria 1 */

Instruções de Carga e Armazenamento

Transferem valores entre o array de variáveis locais e a pilha de operandos de um frame de um método

- carrega uma variável local $\Rightarrow$ pilha de operandos
iload, iload_<n>, lload, lload_<n>, fload, fload_<n>, dload, dload_<n>, aload, aload_<n>
- carrega um valor da pilha de operandos $\Rightarrow$ vetor de variáveis locais
istore, istore_<n>, lstore, lstore_<n>, fstore, fstore_<n>, dstore, dstore_<n>, astore, astore_<n>
- carrega constante na pilha de operandos
bipush, sipush, ldc, ldc_w, ldc2_w, aconst_null, iconst_m1, iconst_<i>, lconst_<l>, fconst_<f>, dconst_<d>

Instruções de Carga e Armazenamento

Estende o índice de uma variável local para 2 bytes

wide <opcode> indexbyte1 indexbyte2

wide iinc indexbyte1 indexbyte2 constbyte1 constbyte2

wide = 196 (0xc4)

Descrição

O <opcode> é um dos seguintes: *iload, fload, aload, lload, dload, istore, fstore, astore, lstore, dstore ou ret.*

Os unsigned bytes *indexbyte1* e *indexbyte2* são montados como índice de 16-bits de uma variável local no frame corrente.

Para as instruções *lload, dload, lstore ou dstore* o índice seguinte (*index + 1*) deve ser um índice para o array de variáveis locais. Os dois apontam para um valor long ou double.

Na segunda forma, os unsigned byte *constbyte1* e *constbyte2* são montados como uma constante com sinal de 16-bit, onde a constante é obtida como $(constbyte1 \ll 8) \mid constbyte2$

Nesse caso, a variável local dado por $(indexbyte1 \ll 8) \mid indexbyte2$ é incrementada pelo valor com sinal dado por $(constbyte1 \ll 8) \mid constbyte2$

Instruções de Carga e Armazenamento

Dado um índice para uma variável local *int*, a carrega na pilha de operandos

iload index

iload = 21 (0x15)

Descrição

O *index* é um **unsigned byte** que deve ser um índice do array de variáveis locais do frame corrente.

A variável local no índice deve conter um **int**. O valor da variável local no índice é empilhado na pilha de operandos.

O opcode **iload** pode ser usado em conjunto com a instrução **wide** para acessar uma variável local usando um índice de 2 bytes

Dado um índice implícito, carrega uma variável local na pilha de operandos

iload_<n>

iload_0 = 26 (0x1a)

iload_1 = 27 (0x1b)

iload_2 = 28 (0x1c)

iload_3 = 29 (0x1d)

Descrição

O *<n>* deve ser um índice no array de variáveis locais do frame corrente. A variável local em *<n>* deve conter um **int**. O valor da variável local em *<n>* é empilhado na pilha de operandos.

Cada instrução **iload_<n>** é a mesma que **iload** com um *index* of *<n>*, exceto que o operando *<n>* é implícito.

Instruções de Carga e Armazenamento

Dado um índice para uma variável local de referência, a carrega na pilha de operandos

aload index

aload = 25 (0x19)

Descrição

O index é um **unsigned byte** que deve ser um índice do array de variáveis locais do frame corrente. A variável local no índice deve conter uma referência. O valor da variável local no índice é empilhado na pilha de operandos.

Essa instrução não pode ser usada para carregar um tipo **returnAddress** de uma variável local na pilha de operandos. Porque?

O opcode **aload** pode ser usado em conjunto com a instrução **wide** para acessar uma variável local usando um **índice de 2 bytes**

Dado um índice implícito, carrega uma variável local na pilha de operandos

aload_<n>

aload_0 = 42 (0x2a)

aload_1 = 43 (0x2b)

aload_2 = 44 (0x2c)

aload_3 = 45 (0x2d)

Descrição

O **<n>** deve ser um índice no array de variáveis locais do frame corrente. A variável local em **<n>** deve conter uma referência. O valor da variável local em **<n>** é empilhado na pilha de operandos.

Cada instrução **aload_<n>** é a mesma que **aload** com um **index** of **<n>**, exceto que o operando **<n>** é implícito.

Instruções de Carga e Armazenamento

Desempilha o int no topo da pilha de operandos no array de variáveis locais, na posição dada por índice para variável local int

istore index

istore = 54 (0x36)

Descrição

O index é um unsigned byte que deve ser um índice do array de variáveis locais do frame corrente. A variável local no índice deve conter um int. Desempilha o int no topo da pilha de operandos no array de variáveis locais, na posição índice.

O opcode istore pode ser usado em conjunto com a instrução wide para acessar uma variável local usando um índice de 2 bytes

Desempilha o int no topo da pilha de operandos no array de variáveis locais, na posição dada por índice para variável local int

istore_<n>

istore_0 = 59 (0x3b)

istore_1 = 60 (0x3c)

istore_2 = 61 (0x3d)

istore_3 = 62 (0x3e)

Descrição

O <n> deve ser um índice no array de variáveis locais do frame corrente. A variável local em <n> deve conter um int. O valor da variável local em <n> é empilhado na pilha de operandos.

Cada instrução *iload_<n>* é a mesma que *iload* com um *index of <n>*, exceto que o operando <n> é implícito.

Instruções de Carga e Armazenamento

Transferem valores entre o array de variáveis locais e a pilha de operandos de um frame de um método

- carrega uma constante $\Rightarrow$ pilha de operandos

bipush, sipush, ldc, ldc_w, ldc2_w, aconst_null, iconst_m1, iconst_<i>, lconst_<l>, fconst_<f>, dconst_<d>

- Comentários

- Instruções com letra entre bicudos denotam famílias e geram instruções específicas com operandos implícitos
 - <i> – int, <l> – long,
 - <f> – float, <d> – double.
 - Formas tipo int são usadas com valores tipo byte, char e short

Carrega Constante na Pilha de Operandos

bipush byte

bipush = 16 (0x10)

- O byte imediato é estendido com sinal para um valor int e empilhado na pilha de operandos

sipush byte1 byte2

sipush = 17 (0x11)

- O valor imediato short (*byte1* << 8) | *byte2* é estendido com sinal para um int e empilhado na pilha de operandos

Carrega Constante na Pilha de Operandos

ldc index

ldc = 18 (0x12)

- O *index* é um unsigned byte que deve ser um índice válido para o pool de constantes da classe corrente. O valor no pool de constante deve ser uma constante do tipo int ou float ou uma referência simbólica para um literal string.
- se do tipo int ou float, o valor numérico da constante é empilhado na pilha de operandos como um int ou float
- se uma referência para uma instância da classe String representando um literal string, uma referência para tal instância é empilhada na pilha de operandos.

Carrega Constante na Pilha de Operandos

ldc_w indexbyte1 indexbyte2

ldc_w = 19 (0x13)

- Os unsigned *indexbyte1* e *indexbyte2* são montados em um índice unsigned de 16-bit do pool de constantes da classe corrente, onde o valor do índice é calculado como $(indexbyte1 \ll 8) \mid indexbyte2$. O valor no pool de constante deve ser uma constante do tipo int ou float ou uma referência simbólica para um literal string.
 - se do tipo int ou float, o valor numérico da constante é empilhado na pilha de operandos como um int ou float
 - se uma referência para uma instância da classe String representando um literal string, uma referência para tal instância é empilhada na pilha de operandos.
- É idêntica à instrução *ldc* exceto que o índice é de 16 bits.

Carrega Constante na Pilha de Operandos

ldc2_w indexbyte1 indexbyte2

ldc2_w = 20 (0x14)

- Os unsigned *indexbyte1* e *indexbyte2* são montados em um índice unsigned de 16-bit do pool de constantes da classe corrente, onde o valor do índice é calculado como $(indexbyte1 \ll 8) \mid indexbyte2$. O valor no pool de constante deve ser uma constante do tipo long ou double.
- o valor numérico da constante é empilhado na pilha de operandos como um long ou double
 - Não existe uma versão ldc2 com um índice de 8 bits.

Carrega Constante na Pilha de Operandos

aconst_null

aconst_null = 1 (0x1)

- Empilha uma referência null para um objeto na pilha de operandos

iconst_<i>

iconst_m1 = 2 (0x2), *iconst_0* = 3 (0x3), *iconst_1* = 4 (0x4)

iconst_2 = 5 (0x5), *iconst_3* = 6 (0x6), *iconst_4* = 7 (0x7)

iconst_5 = 8 (0x8)

- Empilha a constante int <i> (-1, 0, 1, 2, 3, 4 ou 5) na pilha de operandos.
- equivalente a *bipush* <i> para o respectivo valor de <i>, exceto que o operando <i> é implícito.

Carrega Constante na Pilha de Operandos

lconst_<l>

lconst_0 = 9 (0x9)

lconst_1 = 10 (0xa)

- Armazena a constante long 0L ou 1L na pilha de operandos.

fconst_<f>

fconst_0 = 11 (0xb)

fconst_1 = 12 (0xc)

fconst_2 = 13 (0xd)

- Armazena a constante float 0.0, 1.0 ou 2.0 na pilha de operandos.

Carrega Constante na Pilha de Operandos

`dconst_<d>`

`dconst_0 = 14 (0xe)`

`dconst_1 = 15 (0xf)`

- Armazena a constante double 0.0 ou 1.0 na pilha de operandos.

Instruções Aritméticas

Computam um resultado que é tipicamente uma função de dois valores na pilha de operandos

- o resultado é empilhado na pilha de operandos.

Instruções aritméticas

- Valores inteiros
- Valores de ponto flutuantes
- Não há suporte direto para operações aritméticas em valores do tipo **byte**, **short** e **char** ou **boolean**
 - essas operações são manipuladas por instruções de tipos **int**

Instruções Aritméticas

Soma

iadd, ladd, fadd, dadd

Subtração

isub, lsub, fsub, dsub

Multiplação

imul, lmul, fmul, dmul

Divisão

idiv, lddiv, fddiv, dddiv

Resto

irem, lrem, frem, drem

Negate

ineg, lneg, fneg, dneg

Shift

ishl, ishr, iushr, lshl, lshr, lushr.

Bitwise AND

iand, land

Bitwise OR

ior, lor

Bitwise exclusive OR

ixor, lxor

Incremento em variável local

ilnc

Comparação

dcmpg, dcmpl, fcmpg, fcmpl, lcmp

Soma

..., *value1*, *value2* $\Rightarrow$..., *result*
result é calculado como *value1* +
value2

iadd = 96 (0x60)

ladd = 97 (0x61)

fadd = 98 (0x62)

dadd = 99 (0x63)

Multiplicação

..., *value1*, *value2* $\Rightarrow$..., *result*
result é calculado como *value1* *
value2

imul = 104 (0x68)

lmul = 105 (0x69)

fmul = 106 (0x6a)

dmul = 107 (0x6b)

Subtração

..., *value1*, *value2* $\Rightarrow$..., *result*
result é calculado como *value1* -
value2

sub = 100 (0x64)

lsub = 101 (0x65)

fsub = 102 (0x66)

dsub = 103 (0x67)

Divisão

..., *value1*, *value2* $\Rightarrow$..., *result*
result é calculado como *value1* /
value2

idiv = 108 (0x6c)

ldiv = 109 (0x6d)

fdiv = 110 (0x6e)

ddiv = 111 (0x6f)

Resto

Desempilha dois valores do topo da pilha, divide-os como inteiros e empilha o resto

..., *value1*, *value2* $\Rightarrow$..., *result*

- *result* é calculado como $value1 - (value1 / value2) * value2$
 - ocorre truncamento na divisão $value1 / value2$

irem = 112 (0x70)

lrem = 113 (0x71)

frem = 114 (0x72)

drem = 115 (0x73)

A função resto também pode ser aplicada a ponto flutuante

Negação

Desempilha um valor do topo da pilha, nega-o e empilha o resultado

..., *value* $\Rightarrow$..., *result*

- *result* é similar a zero - *value* ou seja, -*value*.

ineg = 116 (0x74)

lneg = 117 (0x75)

fneg = 118 (0x76)

dneg = 119 (0x77)